Assignment 5 - Strategy Pattern

Berdibek Meiirbek | SE-2528 | Option B: Calculator

1. Introduction

I chose a calculator with interchangeable arithmetic operations. `OperationStrategy` declares `execute(left, right)`. Addition, subtraction, and multiplication implement this interface. `Calculator` stores the selected strategy and delegates calculations to it. `Main` selects the algorithms and switches them on the same calculator during one run.

With operands 12 and 3, the program prints:

```
Add: 15.0
Subtract: 9.0
Multiply: 36.0
```

Division is included as a separate extension to demonstrate adding an algorithm without changing existing Java classes.

2. UML class diagram

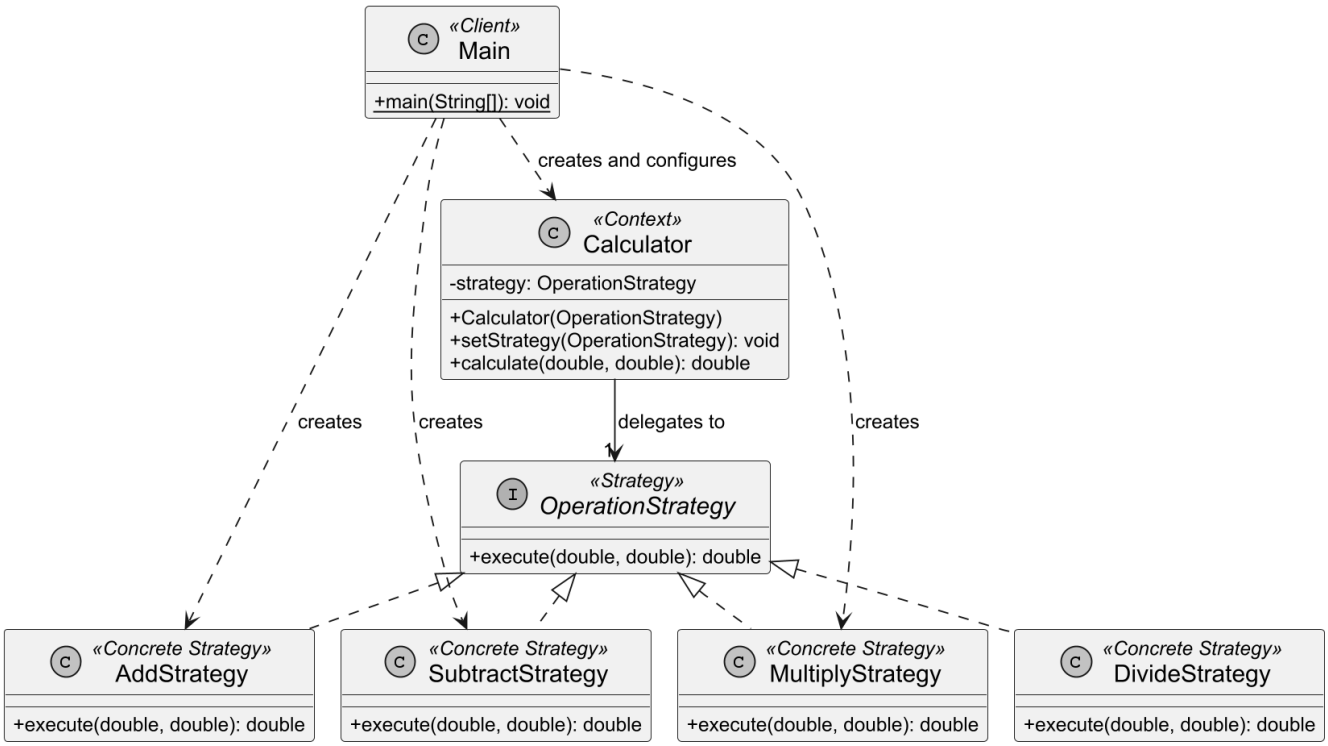


Figure 1. Generated with PlantUML. Diagram source: `uml/strategy.puml`.

`Main` is the Client, `Calculator` is the Context, and `OperationStrategy` is the Strategy interface. `AddStrategy`, `SubtractStrategy`, `MultiplyStrategy`, and `DivideStrategy` are Concrete Strategies. The solid arrow represents the context's stored strategy reference; dashed arrows with hollow triangles represent interface implementation. Main creates the three strategies used in its demonstration. Division can be selected separately through JShell.

3. Clean Code

3.1 Replace operation conditionals with polymorphism

`Calculator.java`:

```
public double calculate(double left, double right) {
    return strategy.execute(left, right);
}
```

This one call replaces an operation-selection chain. The selected object's implementation determines the calculation; the context contains no type checks or operation branches.

3.2 Meaningful names

`Main.java`:

```
calculator.setStrategy(new SubtractStrategy());
```

The class name identifies the operation, and `setStrategy` states that the active algorithm changes. Names such as `left` and `right` identify operand order, which matters for subtraction and division.

3.3 Single Responsibility

`MultiplyStrategy.java`:

```
public class MultiplyStrategy implements OperationStrategy {
    @Override
    public double execute(double left, double right) {
        return left * right;
    }
}
```

This class only multiplies. It does not select algorithms or print output. Each other strategy also has one arithmetic responsibility; output belongs to Main.

3.4 Keep the context independent of concrete algorithms

Selected declarations from `Calculator.java`:

```
private OperationStrategy strategy;

public void setStrategy(OperationStrategy strategy) {
    this.strategy = Objects.requireNonNull(strategy, "strategy");
}
```

The context depends on the interface and accepts any implementation. It imports no concrete strategies. The null check prevents an invalid strategy from replacing the current one.

3.5 Open/Closed Principle: add an algorithm without rewriting existing classes

`DivideStrategy.java`:

```
public class DivideStrategy implements OperationStrategy {
    @Override
    public double execute(double left, double right) {
        if (right == 0) {
            throw new IllegalArgumentException("Cannot divide by zero.");
        }
        return left / right;
    }
}
```

The extension commit adds this class and its entry in `sources.txt`. No existing Java file changes. In JShell, an existing calculator can select it with `calculator.setStrategy(new DivideStrategy())`; calculating 12 and 3 then returns `4.0`. The condition above validates division, rather than selecting a strategy inside the context.

4. Strategy vs. State

Strategy changes the algorithm chosen by a client. State changes behavior according to an object's current state and can coordinate transitions between states. This calculator has alternative arithmetic operations, not a lifecycle such as Draft, Approved, and Closed. Main chooses the operation explicitly; strategies do not switch each other or manage transitions. Therefore, Strategy fits this problem [1, 2].

5. Conclusion

The same calculator can perform different operations while its calculation method stays unchanged. Adding division required a new strategy class and a build-list entry. The trade-off is more small classes than a simple conditional calculator. They keep algorithms separate and make extensions easier to test. Java `double` supports fractional results but has normal floating-point rounding limits.

6. GitHub repository

Mikosh001/Assignment_5_Strategy_Pattern

References and assistance

- Assignment 5 - Strategy Pattern instructions, Astana IT University, 2026-2027.
- [1] Refactoring.Guru: Strategy.
- [2] Refactoring.Guru: State.
- [3] PlantUML: Class diagram.
- OpenAI Codex assisted with code, report preparation, and verification.